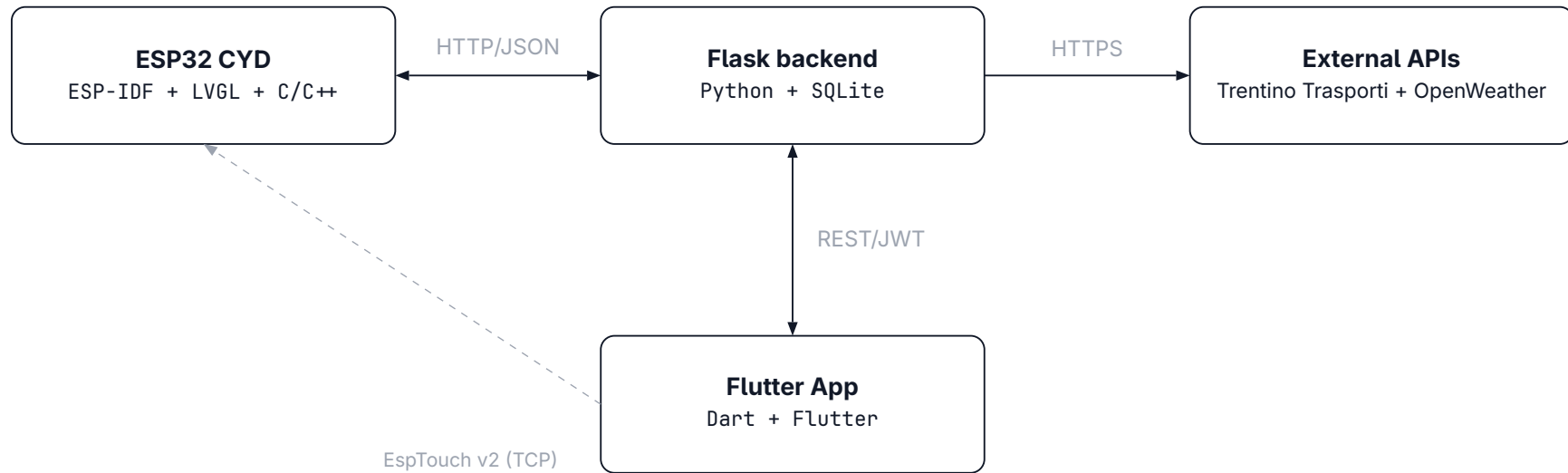


Sdrumo

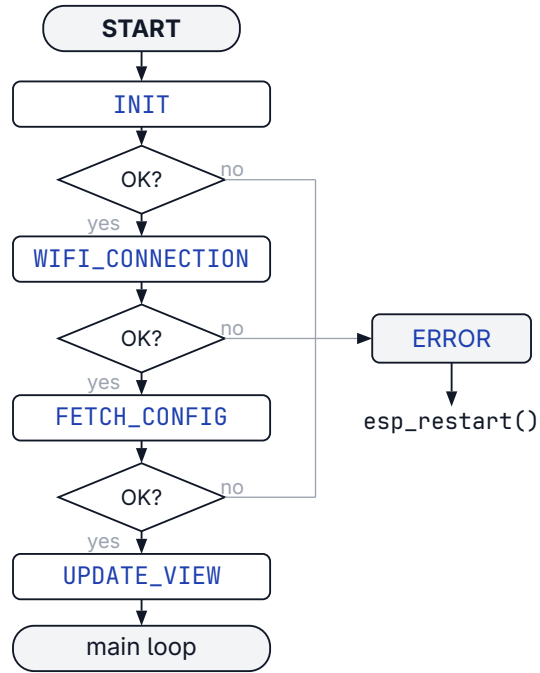
Embedded Software for the Internet of Things

Giulia Cristofolini · Mattia Zagatti · Mirko Lana

Architecture Overview



Software Architecture



C++ singletons

- `Fsm` : state machine
- `Config` : NVS persistence
- `ApiClient` : HTTP client
- `NetHandler` : WiFi + Sntp

FreeRTOS tasks

- `LVGL` port: draw UI on the screen
- `api_poll_task` : bus/weather/config
- `smartconfig_task` : Wi-Fi setup

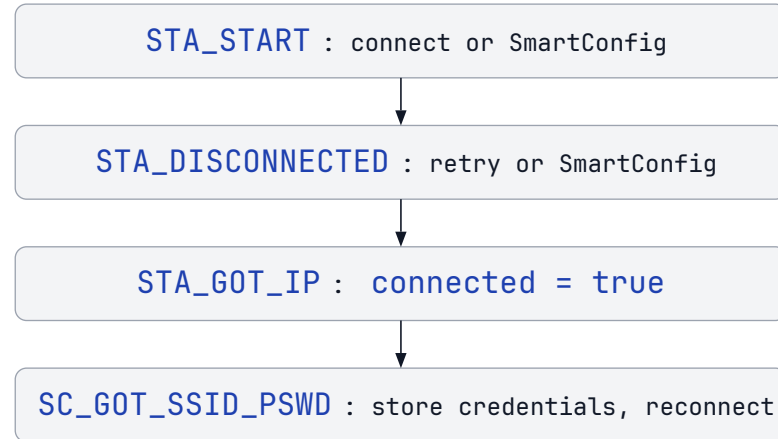
LVGL screens (C)

- `clock` : analog hands, alarm/timer btns
- `bus` : stop picker + trip list (≤ 5)
- `weather` : today + 3-day forecast
- `alarm`, `timer` : scrollable pickers
- `pairing` : token display + done btn

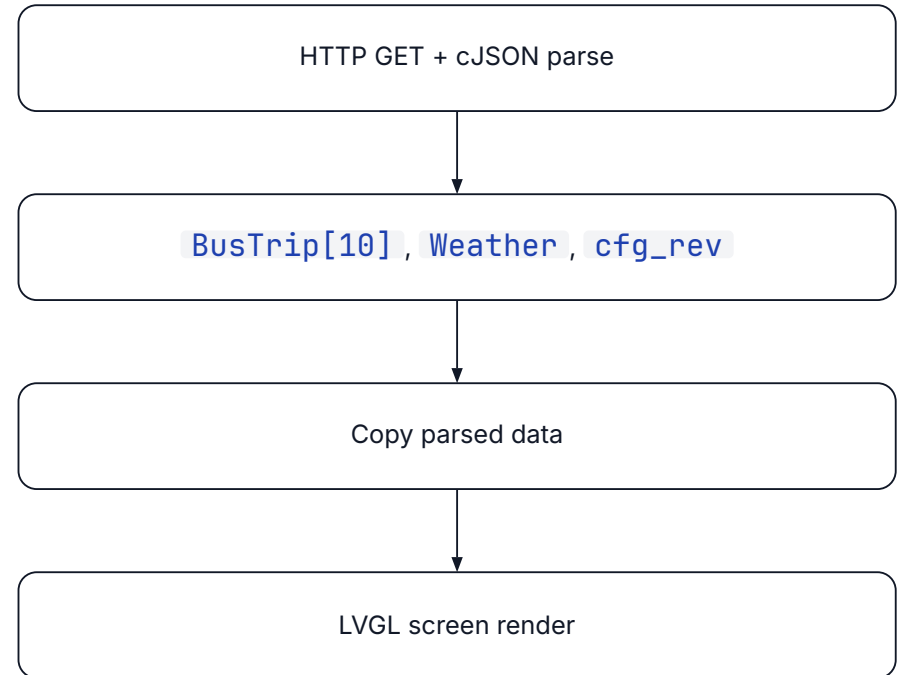
HW/SW Interaction & Data Flow

WiFi events

All hardware events dispatched by ESP-IDF event loop to a single callback. SPI DMA interrupts handled by drivers; app registers only callbacks.



Data processing chain



Representative Code

WiFi Event Handler (`net.cpp`)

```
void NetHandler::event_handler(  
    void *arg, esp_event_base_t event_base,  
    std::int32_t event_id, void *event_data) {  
    static NetHandler &net =  
NetHandler::get_instance();  
    static Config &config =  
Config::get_instance();  
  
    if (event_base == WIFI_EVENT  
        && event_id == WIFI_EVENT_STA_START) {  
        if (net.smartconfig_running) return;  
        if (config.get_ssid()[0U] != '\0'  
            && config.get_password()[0U] != '\0')  
            esp_wifi_connect();  
        else  
            net.start_smartconfig();  
        return;  
    }  
    if (event_base == WIFI_EVENT  
        && event_id ==
```

LVGL Screen Manager (`screen_manager.c`)

```
void ui_load_screen(lv_obj_t *screen) {  
    screen_clock_destroy();  
    screen_bus_destroy_timer();  
    screen_pairing_destroy_timer();  
    lv_obj_clean(screen);  
  
    switch (present_screen_type) {  
        case SCREEN_BOOT:  
            ui_load_screen_boot(screen); break;  
        case SCREEN_WIFI:  
            ui_load_screen_wifi(screen); break;  
        case SCREEN_CLOCK:  
            ui_load_screen_clock(screen);  
            ui_load_arrows_btn(screen); break;  
        case SCREEN_BUS:  
            ui_load_screen_bus(screen);  
            ui_load_arrows_btn(screen); break;  
        case SCREEN_ALARM:  
            ui_load_screen_alarm(screen); break;  
        case SCREEN_TIMER:
```

```

WIFI_EVENT_STA_DISCONNECTED) {
    xEventGroupClearBits(net.event_group,
                        NET_CONNECTED_BIT);
    if (net.retry_count < NET_MAX_RETRY)
        { ++net.retry_count;
    esp_wifi_connect(); }
    else net.start_smartconfig();
    return;
}
if (event_base == IP_EVENT
    && event_id == IP_EVENT_STA_GOT_IP) {
    net.connected = true; net.retry_count =
0;

    xEventGroupSetBits(net.event_group,
                        NET_CONNECTED_BIT);
    if (net.came_from_smartconfig)
        net.came_from_smartconfig = false;
    return;
}
if (event_base == SC_EVENT
    && event_id == SC_EVENT_SCAN_DONE)
return;
if (event_base == SC_EVENT
    && event_id == SC_EVENT_FOUND_CHANNEL)
return;

```

```

        ui_load_screen_timer(screen); break;
case SCREEN_PAIRING:
    ui_load_screen_pairing(screen);
break;
case SCREEN_WEATHER:
    ui_load_screen_weather(screen);
    ui_load_arrows_btn(screen); break;
default:
    ui_load_screen_wifi(screen);
    ui_load_arrows_btn(screen); break;
    }
}

```

```

if (event_base == SC_EVENT
    && event_id == SC_EVENT_GOT_SSID_PSWD) {
    auto *evt = static_cast<
        smartconfig_event_got_ssid_pswd_t *>(
            event_data);
    char ssid[33U] = { 0 };
    char password[65U] = { 0 };
    memcpy(ssid, evt->ssid, sizeof(evt-
>ssid));
    memcpy(password, evt->password,
        sizeof(evt->password));
    std::size_t sl = strlen(ssid,
sizeof(ssid)-1);
    std::size_t pl = strlen(password,
sizeof(password)-1);
    Result cr = config.set_credentials(
        std::string_view(ssid, sl),
        std::string_view(password, pl));
    if (cr != Result::SUCCESS) {
    } else if (config.store() !=
Result::SUCCESS) {}
    if (evt->type == SC_TYPE_ESPTOUCH_V2) {
        uint8_t rvd[RVD_DATA_SIZE] = {};
        ESP_ERROR_CHECK(
            esp_smartconfig_get_rvd_data(

```

```

        rvd, sizeof(rvd)));
    }
    wifi_config_t cfg = {};
    memcpy(cfg.sta.ssid, evt→ssid,
           sizeof(cfg.sta.ssid));
    memcpy(cfg.sta.password, evt→password,
           sizeof(cfg.sta.password));
    net.retry_count = 0;
    ESP_ERROR_CHECK(
        esp_wifi_set_config(WIFI_IF_STA,
&cfg));
    ESP_ERROR_CHECK(esp_wifi_connect());
    return;
}
if (event_base == SC_EVENT
    && event_id == SC_EVENT_SEND_ACK_DONE) {
    xEventGroupSetBits(net.event_group,
NET_ESPTOUCH_DONE_BIT);
    return;
}
}

```


Testing & Conclusions

Testing

- `idf.py monitor` : FSM states, HTTP payloads, task logs
- Bruno collection: testing all APIs before their implementation in the firmware
- Physical device: checking if actual result = desired result
- `flutter analyze` + Android/Linux run
- `idf.py erase-flash` : full re-registration and re-pairing + Wi-Fi configuration

Result

Working embedded product: live busses + weather on a 2.8" touch display; zero-config WiFi pairing.

Future improvements

- HTTPS support
- MQTT / WebSocket to replace config polling
- Light-sleep + screen off during night
- Alarm & timer backend (screens already done)
- Buzzer for timer and alarm